UADE

FUNDAMENTOS DE INFORMÁTICA

Métodos de Búsqueda en Listas

Búsqueda secuencial y búsqueda binaria

Clase 11 · Departamento de Tecnología Informática (DEINI)

11

AGENDA

Temas de la clase

1. ¿Qué es buscar en una lista?
2. Búsqueda secuencial
3. Búsqueda binaria
4. Comparación entre ambos métodos
5. Ejercicios

11

UNIDAD 6

¿Qué es Buscar en una Lista?

- Dado un valor y una lista de elementos, un algoritmo de búsqueda determina si ese valor está presente y, si lo está, en qué posición.
- Es una de las operaciones más frecuentes al trabajar con listas: validar datos, evitar duplicados, recuperar información, etc.
- Existen distintas estrategias según si la lista está ordenada o no.

11

TAMBIÉN LLAMADA BÚSQUEDA LINEAL

Búsqueda Secuencial

- Recorre la lista elemento por elemento, comparando cada uno con el valor buscado.
- Termina cuando encuentra el valor, o cuando llega al final de la lista sin encontrarlo.
- No requiere que la lista esté ordenada: funciona con cualquier lista.

11

PYTHON

Búsqueda secuencial en Python

```
def busqueda_secuencial(lista, valor):
    for i in range(len(lista)):
        if lista[i] == valor:
            return i   # posición encontrada
    return -1   # no está en la lista

numeros = [4, 8, 15, 16, 23, 42]
print(busqueda_secuencial(numeros, 16))   # 3
print(busqueda_secuencial(numeros, 99))   # -1
```

11

ANÁLISIS

Complejidad de la Búsqueda Secuencial

- En el peor caso (el valor no está, o está al final) hay que recorrer todos los n elementos: complejidad O(n).
- Simple de implementar y funciona siempre, pero puede ser lenta en listas muy grandes.

11

REQUIERE LISTA ORDENADA

Búsqueda Binaria

- Requiere que la lista esté previamente ordenada.
- Compara el valor buscado con el elemento central de la lista:
- Si coinciden
  - se encontró el valor.
- Si el valor buscado es menor
  - se repite la búsqueda en la mitad izquierda.
- Si el valor buscado es mayor
  - se repite la búsqueda en la mitad derecha.
- En cada paso se descarta la mitad de los elementos restantes.

11

PYTHON

Búsqueda binaria en Python

```python
def busqueda_binaria(lista, valor):
    izq, der = 0, len(lista) - 1
    while izq <= der:
        medio = (izq + der) // 2
        if lista[medio] == valor:
            return medio
        elif lista[medio] < valor:
            izq = medio + 1
        else:
            der = medio - 1
    return -1

numeros = [4, 8, 15, 16, 23, 42]   # ya ordenada
print(busqueda_binaria(numeros, 23))   # 4
```

11

PYTHON · SEGUIMIENTO

Búsqueda binaria paso a paso

```python
def busqueda_binaria(lista, valor):
    izq, der = 0, len(lista) - 1
    iteraciones = 0
    while izq <= der:
        medio = (izq + der) // 2
        iteraciones += 1
        print(f"izq={izq} der={der} medio={medio} lista[medio]={lista[medio]}")
        if lista[medio] == valor:
            return medio, iteraciones
        elif lista[medio] < valor:
            izq = medio + 1
        else:
            der = medio - 1
    return -1, iteraciones

numeros = [4, 8, 15, 16, 23, 42]   # ya ordenada
pos, pasos = busqueda_binaria(numeros, 23)
print("posición:", pos, "- iteraciones:", pasos)
```

SALIDA

```
izq=0 der=5 medio=2 lista[medio]=15
izq=3 der=5 medio=4 lista[medio]=23
posición: 4 - iteraciones: 2
```

Para probar en clase

- Buscar el 5 (no está): ¿qué pasa con izq y der?
- Usar una lista desordenada: ¿lo encuentra siempre?

11

ANÁLISIS

Complejidad de la Búsqueda Binaria

- Cada paso descarta la mitad de los elementos restantes: complejidad O(log n).
- Mucho más rápida que la secuencial en listas grandes.
- Costo: exige mantener la lista ordenada (por ejemplo, con un algoritmo de ordenamiento previo).

11

¿DE DÓNDE SALE log n?

Juego: adiviná el número

- Pienso un número del 1 al 100. Ustedes preguntan un número y yo sólo respondo «mayor» o «menor».
- ¿Cuántas preguntas necesitan, como máximo, para adivinarlo?
- La mejor estrategia: preguntar siempre **el del medio** (50, después 25 o 75, …). Cada respuesta descarta la mitad.
- ¡Eso es exactamente lo que hace la búsqueda binaria!

Ejemplo buscando el 37

50 *menor* **25** *mayor* **37** *¡es!*

7

preguntas alcanzan siempre para adivinar un número entre 1 y 100.

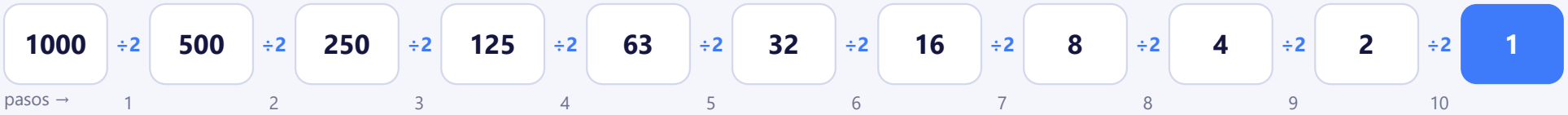
Con 1 a 1000 alcanzan 10.

11

¿DE DÓNDE SALE log n?

¿Cuántas veces puedo dividir por 2?

Empiezo con una lista de 1000 elementos. En cada paso de la búsqueda binaria me queda la mitad:



≈ 10

divisiones por 2 para pasar de 1000 elementos a uno solo.

La pregunta clave

¿Cuántas veces puedo partir n por la mitad hasta que me quede 1?

Esa cantidad de pasos es lo que tarda la búsqueda binaria en el peor caso.

11

¿DE DÓNDE SALE log n?

De dividir a multiplicar: el logaritmo

Miremos la misma pregunta al revés: empiezo en 1 y voy duplicando.

1 ×2 2 ×2 4 ×2 8 ×2 16 ×2 32 ×2 64 ×2 128 ×2 256 ×2 512 ×2 **1024**

10 duplicaciones: $2^{10} = 1024 \approx 1000$

A esa pregunta se le pone nombre

«¿A qué número tengo que elevar el 2 para llegar a n?»

Eso es el **logaritmo en base 2**. O, dicho más simple: **cuántas veces puedo dividir n por 2.**

$$\log_2(1000) \approx 10$$

porque $2^{10} = 1024$

11

¿DE DÓNDE SALE log n?

¿Cuánto crece cada búsqueda?

Cantidad máxima de pasos según el tamaño de la lista:

Elementos (n)	Secuencial (n)	Binaria ($\log_2 n$)
1.000	1.000	**10**
1.000.000	1.000.000	**20**
1.000.000.000	1.000.000.000	**30**

- Duplicar la lista agrega a la binaria un solo paso: es un corte más.
- La notación formal O(log n) la van a ver más adelante; acá alcanza con «crece muy lento».

+10

pasos más necesita la binaria si la lista es 1000 veces más grande.

La secuencial necesita 1000 veces más.

11

PYTHON · EXPERIMENTO

Comprobarlo con código

```python
def busqueda_secuencial(lista, valor):
    pasos = 0
    for i in range(len(lista)):
        pasos += 1
        if lista[i] == valor:
            return i, pasos
    return -1, pasos

# busqueda_binaria: la de antes, sin el print de seguimiento

for n in [1000, 1000000]:
    numeros = list(range(1, n + 1))   # ordenada
    pos, p_sec = busqueda_secuencial(numeros, n - 1)
    pos, p_bin = busqueda_binaria(numeros, n - 1)
    print(n, "elementos -> secuencial:", p_sec, "| binaria:", p_bin)
```

SALIDA

```
1000 elementos ->
  secuencial: 999 | binaria: 9
1000000 elementos ->
  secuencial: 999999 | binaria: 19
```

¿Qué observamos?

- La lista creció 1000 veces y la binaria pasó de 9 a 19 pasos.
- Los números coinciden con la tabla: la teoría se cumple en la práctica.

11

RESUMEN

Búsqueda Secuencial vs. Binaria

	Secuencial	Binaria
Requiere lista ordenada	No	**Sí**
Complejidad	O(n)	**O(log n)**
Implementación	Muy simple	**Simple**
Conviene usarla cuando…	la lista es chica o no está ordenada	**la lista es grande y ya está ordenada**

11

PARA RESOLVER

Ejercicios Propuestos

- Implementar búsqueda_secuencial y probarla con una lista de nombres (strings).
- Implementar búsqueda_binaria y probarla con una lista de números ordenada.
- Modificar la búsqueda secuencial para contar cuántas comparaciones realiza.
- Dada una lista desordenada, ordenarla primero (usando sorted()) y luego aplicar búsqueda binaria.
- Probar busqueda_binaria con una lista desordenada: ¿encuentra siempre el valor? ¿Por qué?
- Escribir una función que indique si conviene usar búsqueda secuencial o binaria según el tamaño y el estado (ordenada o no) de la lista.

¿Preguntas?

Próxima clase: Métodos de ordenamiento de listas: selección, burbuja e inserción